

Age of Information Minimization on Graphs with Graph-Aware Reinforcement Learning (Final Solution and Extended Report)

Roman Iashchenko and Manyou Ma

Abstract—We study Age of Information (AoI) minimization in a graph-based status update system under a one-action-per-step constraint. We build an end-to-end simulator with stochastic refresh dynamics, compare classical scheduling heuristics, and develop graph-aware deep reinforcement learning via a GNN-based DQN (GNN-DQN). We evaluate across multiple graph families (line, star, Erdős-Rényi), analyze generalization gaps, and propose a final solution that prioritizes robustness on unseen topologies. To satisfy an extended-review format, we include implementation details, evaluation protocol, and code excerpts.

Index Terms—Age of Information, Reinforcement Learning, Deep Q-Network, Graph Neural Networks, Scheduling, Generalization

I. INTRODUCTION

Age of Information (AoI) is a freshness metric: it measures how long ago the most recently received update was generated. AoI optimization appears in IoT monitoring, telemetry, caching, and distributed status update systems, where one typically faces limited bandwidth or a constrained number of updates per time step. A key practical feature is that the system is *networked*: an update to one component may affect other components (directly or indirectly), hence topology matters.

This project studies AoI minimization on graphs under a strict constraint: at each time step, the controller can attempt to refresh *only one* node. We treat this as a Markov decision process (MDP), implement a simulator, and compare: (i) robust heuristics (random, stale-first greedy, degree-weighted), (ii) value-based deep RL (DQN), (iii) a graph-aware variant (GNN-DQN). The final outcome is a robustness-first policy recommendation supported by experimental evidence.

II. BACKGROUND: AGE OF INFORMATION

A. AoI as a freshness metric

In continuous time, AoI at a receiver increases linearly and drops when a new update is received. In discrete time, we use an equivalent concept: each node has an age value $a_i(t) \in \mathbb{R}_{\geq 0}$ that increases each step unless refreshed. The canonical objective is to minimize long-run average AoI, commonly approximated in finite-horizon simulation by average AoI over an episode.

Roman Iashchenko is with HSE University, Faculty of Computer Science, Moscow, Russia (email: roma.yashchenko3@gmail.com).

Manyou Ma is with the School of Artificial Intelligence, Shenzhen Technology University, Shenzhen, China (email: mamanyou@sztu.edu.cn).

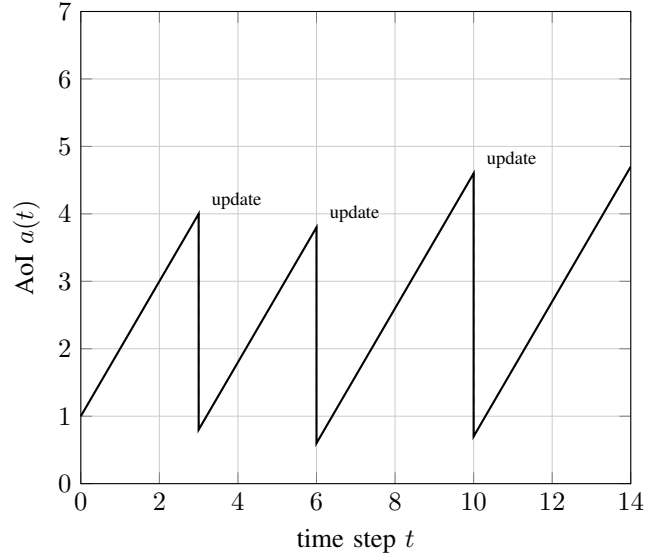


Fig. 1: Sawtooth intuition: AoI increases over time and drops when an update succeeds.

B. Sawtooth intuition

Fig. 1 illustrates the typical “sawtooth” behavior of AoI for a single node.

III. PROBLEM SETUP AS AN MDP

A. Graph and state

Let $G = (V, E)$ be an undirected graph with $|V| = N$ nodes. Each node $i \in V$ has an AoI value $a_i(t) \in \mathbb{R}_{\geq 0}$ at time t . The MDP state is the AoI vector:

$$s_t = (a_1(t), \dots, a_N(t)) \in \mathbb{R}_{\geq 0}^N. \quad (1)$$

B. Action and constraint

At each step the agent chooses exactly one node to refresh:

$$u_t \in \{1, \dots, N\}. \quad (2)$$

This is the “one-action-per-step” scheduling constraint.

C. Stochastic transition model

We use a discrete-time simulator with stochastic refresh success probability p . A refresh attempt succeeds with probability p . On success, the chosen node is reset to 0. In the *graph-based* variant, neighbors may also benefit (“diffusion-like” effect) via an additional stochastic rule described in Section V.

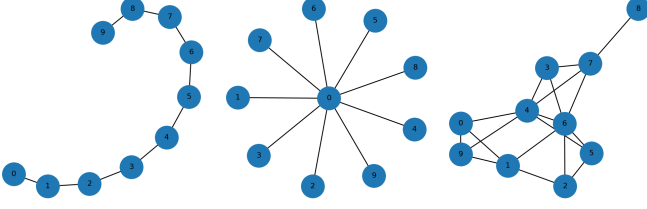


Fig. 2: Illustrations of evaluated graph families (line, star, Erdős–Rényi).

D. Reward and objective

We use mean AoI as the cost and its negative as the reward:

$$r(t) = -\frac{1}{N} \sum_{i=1}^N a_i(t). \quad (3)$$

Thus maximizing expected return corresponds to minimizing average AoI. Each episode lasts a fixed horizon T .

E. Evaluation metric

For an episode of length T , we report

$$\text{MeanAoI} = \frac{1}{T} \sum_{t=1}^T \left(\frac{1}{N} \sum_{i=1}^N a_i(t) \right). \quad (4)$$

Final results are averaged over multiple episodes and random seeds, reporting mean and standard deviation.

IV. GRAPH FAMILIES AND EXPERIMENTAL SCOPE

A. Graph families

We evaluate on three representative graph families (all with $N = 10$):

- **Line (path)** graph.
- **Star** graph (one hub connected to all leaves).
- **Erdős–Rényi** random graphs $G(N, p_e)$.

In generalized training we sample a new Erdős–Rényi graph each episode from a fixed distribution p_e . For testing we also use an unseen ER set (denoted `erdos_new`) generated with different seeds.

B. Why these families

These families cover distinct structural regimes: line graphs have local neighborhoods and no dominant hub, star graphs have extreme centrality, and ER graphs capture diverse random connectivity patterns. This mix is useful to stress-test whether a policy is topology-aware and whether it generalizes beyond its training distribution.

V. SIMULATOR

A. Environment interface

The simulator follows a Gym-style API: `reset()` initializes a graph and AoI state, and `step(action)` applies one update attempt and returns next state and reward.

B. State dynamics

We use the following discrete-time logic (one concrete variant used in our experiments):

- 1) Ages increase by $+1$ for all nodes.
- 2) With probability p , the selected node $u(t)$ is refreshed and its AoI resets to 0.
- 3) Optionally, neighbors of $u(t)$ can also be refreshed stochastically (“diffusion-like” effect). In our simplified model, each neighbor refresh succeeds independently with probability p after the main node succeeds.

This design is intentionally simplified: it captures (i) stochastic delivery and (ii) topology-coupled benefit, without modeling queues, variable packet delays, or multi-hop protocols.

C. Trajectory generation

We generate trajectories for baselines and learned agents for later analysis. A trajectory stores $(s_t, a_t, r_t, s_{t+1}, \text{done})$ at each time step, enabling offline evaluation and debugging.

VI. BASELINES

We compare against three classical heuristics:

- **Random**: select a node uniformly at random.
- **Greedy (stale-first)**: select $\arg \max_i a_i(t)$.
- **Degree-weighted**: select $\arg \max_i (\deg(i) \cdot a_i(t))$.

A. Why degree-weighted is a strong candidate

Degree is a simple proxy for how widely an update can influence the network in diffusion-like dynamics. If a high-degree node is stale, refreshing it can reduce AoI not only locally (for the node itself) but also indirectly (via neighbor refresh). This makes the policy robust in random graphs where local structure varies across nodes and episodes.

VII. RL METHODS

A. DQN on vector state

A standard DQN approximates action-values $Q_\theta(s, i)$, mapping the AoI vector state to per-node Q-values. This can be effective on a fixed graph but does not explicitly encode graph structure; topology must be implicitly learned from experience, which is difficult to generalize.

B. GNN-DQN: graph-structured value function

To incorporate topology, we represent the state as node features and use a message-passing GNN to compute per-node Q-values. Each node i has a feature vector x_i (in our experiments: AoI plus degree-related signal). Given a graph G and node features X , the GNN produces embeddings h_i by aggregating neighborhood information, then outputs Q-values $Q_\theta(s, i)$.

C. Training regimes: fixed vs generalized

We use two regimes:

- **Fixed-graph training**: train and evaluate on one graph family/instance.
- **Generalized training**: sample a new ER graph each episode and aim to generalize to unseen graphs (line/star/ER with new seeds).

TABLE I: Key simulation and training parameters

Parameter	Value
Nodes N	10
Episode horizon T	50
Success probability p	0.8
Discount γ	0.99
Learning rate	1×10^{-3}
Hidden size	64
Batch size	64
Replay buffer size	20000
Warmup	500 steps
Target update	every 200 steps
Exploration	ϵ -greedy (multiplicative decay)

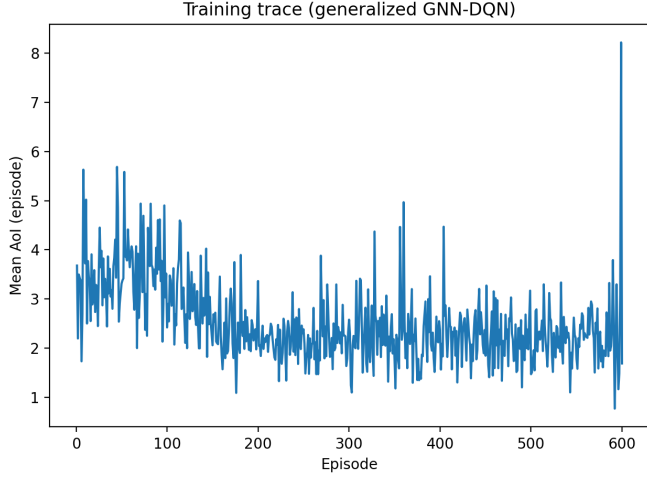


Fig. 3: Training trace (generalized GNN-DQN): episodic mean AoI vs episode index.

D. Optimization details

We use a replay buffer, target network, and ϵ -greedy exploration. The TD target is computed using the target network, and the Q-network is trained by minimizing squared TD error.

VIII. EXPERIMENTAL SETUP

A. Hyperparameters and protocol

Table I summarizes the main hyperparameters.

B. Reproducibility

Experiments are repeated across multiple random seeds for both environment generation and stochastic transitions. We report mean and standard deviation of MeanAoI across seeds. Implementation is provided in the accompanying repository (environment, baselines, training loops, and evaluation scripts).

IX. RESULTS

A. Training trace

Fig. 3 shows episodic mean AoI during generalized training (lower is better).

B. Final performance across families

Fig. 4 aggregates final performance across graph families.

TABLE II: Example numbers (mean AoI; lower is better)

Policy	Line	Star	erdos_new
Random	4.182	5.460	3.234
Greedy	2.823	4.700	2.494
Degree-weighted	2.733	1.765	2.089
GNN-DQN (final run)	3.920	0.579	7.317

C. Numerical snapshot

Table II reports one concrete evaluation snapshot.

D. Policy diagnostics

We inspect action histograms for learned policies. This helps detect collapse (repeatedly selecting a single node independent of state).

X. ANALYSIS AND DISCUSSION

A. What worked

Degree-weighted is consistently strong on random graphs, suggesting degree is a robust structural signal under diffusion-like dynamics. **GNN-DQN** can achieve excellent performance on star graphs, often learning to prioritize the hub, which is consistent with the dominant-centrality nature of the topology.

B. Where learning struggled: generalization

The key difficulty is robust generalization to unseen random graphs. We observed that a learned policy can: (i) overfit to patterns specific to the training distribution, (ii) collapse to a narrow subset of actions, (iii) exhibit high variance across seeds on *erdos_new*. These failure modes are directly visible in action histograms and in increased variance of the evaluation metric.

C. Why star is “easy”

In a star graph, there is a single obvious structural bottleneck: the hub. Refreshing the hub frequently is close to optimal because it directly connects to all leaves and can dominate diffusion-like benefits. This makes the mapping from state to action easier to learn and easier to stabilize.

D. Why ER is “hard”

In ER graphs, local structures vary: there is no universal “best” node. The optimal action depends on the current AoI pattern and local connectivity. A model that does not reliably encode these local patterns (or that collapses) will degrade quickly.

XI. FINAL SOLUTION (ROBUSTNESS-FIRST)

A. Design principle

The committee-facing version of the conclusion should be simple: *when generalization is uncertain, prefer a robust heuristic backbone; use RL only when validated.*

B. Final recommended policy

We propose a hybrid final solution:

- Detect whether a graph is *hub-dominated* (star-like) by checking if $\max_i \deg(i)$ is significantly larger than the mean degree.
- If hub-dominated: prioritize the hub (optionally combined with stale-first behavior).
- Otherwise: use degree-weighted $\arg \max_i \deg(i) \cdot a_i(t)$ as the default.
- Optionally: use a learned GNN policy only if it passes a validation gate on unseen graphs (low variance, no collapse).

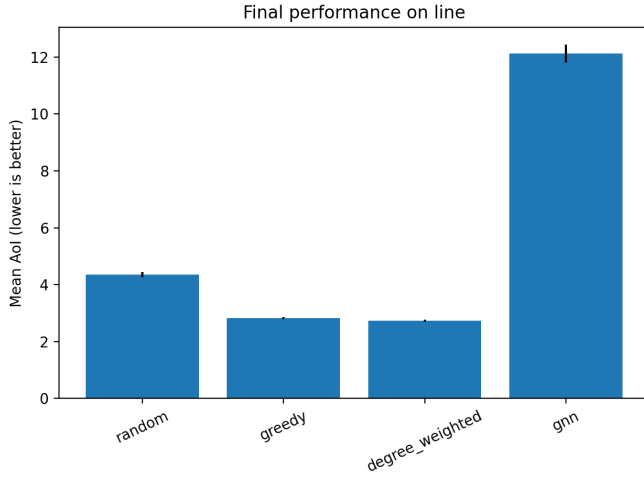
C. Hybrid rule diagram (drawn in LaTeX)

D. Why this is a reasonable “final answer”

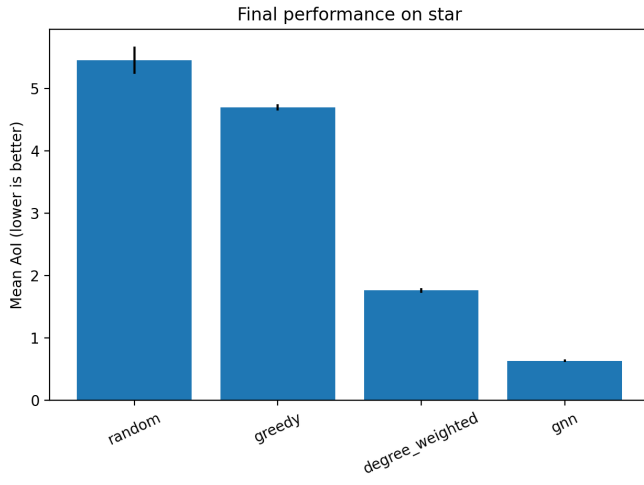
This policy is: (i) explainable, (ii) cheap to compute, (iii) stable across graph families, (iv) consistent with empirical observations that degree-weighted is robust on ER graphs while hub prioritization is strong on star-like graphs.

XII. CONCLUSION

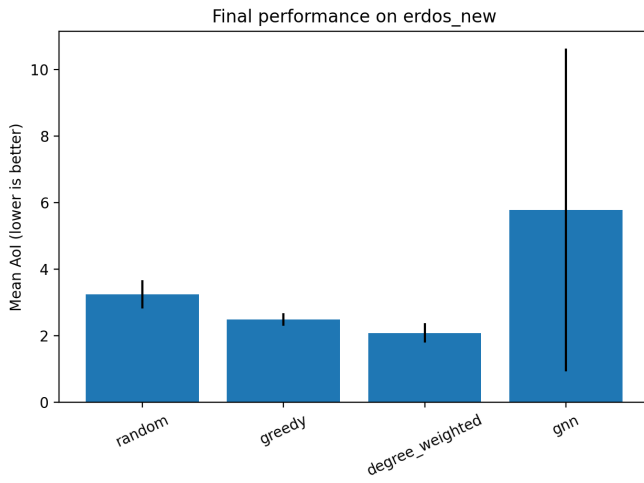
We implemented a graph-based AoI simulator, evaluated classical heuristics, and trained graph-aware RL agents. Empirically, the degree-weighted heuristic is the most robust across unseen random graphs, while GNN-DQN can be extremely strong on hub-dominated topologies. The final recommended solution prioritizes reliability across graph families and uses learning only under validation.



(a) Line graphs.

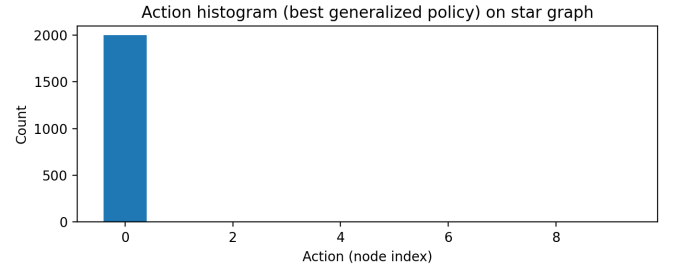


(b) Star graphs.

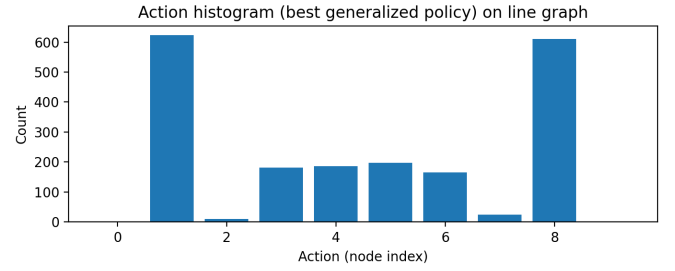


(c) Unseen ER graphs (erdos_new).

Fig. 4: Final mean AoI across graph families (lower is better).



(a) Star graph.



(b) Line graph.

Fig. 5: Action histograms for the learned policy.

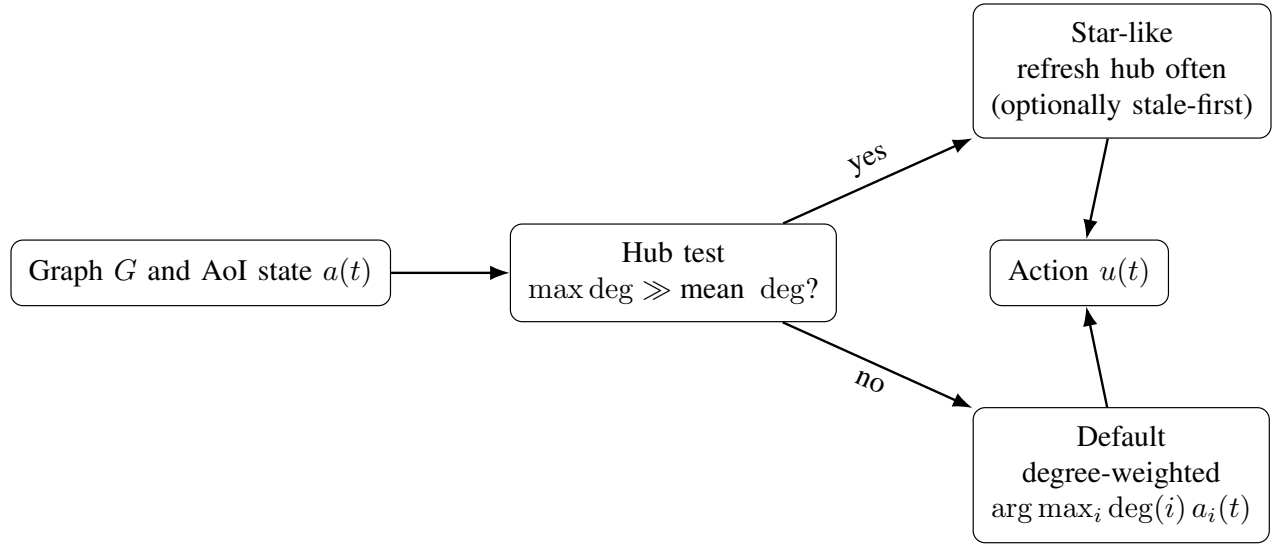


Fig. 6: Final robustness-first solution: detect star-likeness, otherwise use degree-weighted as a reliable backbone.

APPENDIX A IMPLEMENTATION EXCERPTS (EXTENDED REVIEW)

A. Environment step (schematic)

```
def step(action):
    ages = ages + 1
    if bernoulli(p):
        ages[action] = 0
        for nb in neighbors[action]:
            if bernoulli(p):
                ages[nb] = 0
    reward = -ages.mean()
    return ages, reward, done, info
```

B. Baselines

```
def greedy_policy(ages):
    return int(ages.argmax())

def degree_weighted_policy(ages, deg):
    score = deg * ages
    return int(score.argmax())
```

C. GNN forward pass (schematic)

```
def forward(data):
    x, edge_index = data.x, data.edge_index
    h = gcn1(x, edge_index).relu()
    h = gcn2(h, edge_index).relu()
    q = head(h).squeeze(-1)
    return q
```

D. DQN training loop (schematic)

```
for episode in range(E):
    s = env.reset()
    for t in range(T):
        a = eps_greedy(q_net, s)
        ns, r, done, _ = env.step(a)
        replay.push(s, a, r, ns, done)
        s = ns
        if replay.ready():
            batch = replay.sample()
            loss = td_loss(q_net, target_net, batch)
            opt.step(loss)
        if done: break
    maybe_update_target()
```

REFERENCES

- [1] S. Kaul, M. Gruteser, V. Rai, and J. Kenney, “Real-time status: How often should one update?,” in *Proc. IEEE INFOCOM*, 2012.
- [2] R. D. Yates, Y. Sun, D. R. Brown, S. K. Kaul, E. Modiano, and S. Ulukus, “Age of information: An introduction and survey,” *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 5, pp. 1183–1210, 2021.
- [3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [4] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [5] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. ICLR*, 2017.